
Machine Learning Based Analog Circuit Fault Diagnosis
PDF 16 — Chapter 23: Complete Python Source Code
Project Folder Structure | [day12_build_sweep_dataset.py](#) Explained Line-by-Line

PART 1 — Project Folder Structure & Python Files

Chapter 23 — Complete Python Source Code Explained Line-by-Line Part 1 — Project Folder Structure & Python Files Before Starting Congratulations. Until now, we have studied the project from an engineering perspective. Now we'll study the actual code. This chapter is different. We are not learning Machine Learning. We are learning how the entire software was written. Goal of This Chapter After completing

Chapter 23,

you should be able to answer questions like "Why did you import NumPy?" "Why did you use Pandas?" "What does this function do?" "Why is this variable needed?" "How do all Python files communicate?" without looking at the code. Before Opening the Code Most students open Python files and immediately start reading line 1. That is the wrong approach. First, understand the project structure.

Complete Project Structure

Conceptually, our project looks like this. Analog_Fault_Diagnosis/ | ├── dataset/ | dataset.csv | ├── feature_extraction.py | ├── train_svm.py | ├── train_ann.py | ├── predict.py | ├── dashboard.py | ├── saved_model.pkl | ├── scaler.pkl | └── requirements.txt Your actual filenames may differ, but the architecture is almost the same. What Does Each File Do? Let's study them one by one. File 1 Dataset dataset.csv Question. What is this? Simply our data. Contains 17 features Fault Label. Nothing else.

PROFESSOR ASKS

What is stored inside dataset.csv?

EXCELLENT ANSWER

The dataset file stores the extracted numerical features and their corresponding fault labels used for training and evaluating the machine learning models. File 2 feature_extraction.py Question. What happens here? Everything related to waveform processing. Inside this file, functions calculate Peak RMS Mean Energy

Rise Time

Settling Time

DC Level and all remaining features. Pipeline becomes Waveform

Feature Extraction

17 Features

PROFESSOR ASKS

What is the purpose of feature_extraction.py?

EXCELLENT ANSWER

This file converts the raw waveform into the 17-dimensional numerical feature vector required by the machine learning classifier. File 3 train_svm.py This file contains everything related to training the SVM. Inside you usually find

Read Dataset

Split Data

Scale Features

GridSearchCV — Train SVM.

Save Model

Notice training only. No dashboard. No prediction.

PROFESSOR ASKS

What is train_svm.py responsible for?

EXCELLENT ANSWER

It performs preprocessing, trains the Support Vector Machine using the extracted features, optimizes hyperparameters, evaluates performance, and saves the trained model. File 4 train_ann.py Almost the same idea. Instead of SVM, this file creates the ANN. Inside you usually find

Create Network

Compile — Train. Evaluate

Save Model

Question. Does this file extract features? No. It only trains the ANN. File 5 predict.py This file is used after training. Question. What does it do? Very simple.

Load Model

Load Scaler

Receive Features

Predict That's all.

PROFESSOR ASKS

Why separate prediction from training?

EXCELLENT ANSWER

Separating prediction from training improves modularity, avoids unnecessary retraining, and allows the trained model to be reused efficiently. File 6 dashboard.py This is the GUI. Question. Does dashboard.py contain Machine Learning? No. It calls other modules. Its job is Buttons

File Upload

Call Feature Extraction

Call Prediction

Display Results

Think of it as the manager.

PROFESSOR ASKS

What is dashboard.py responsible for?

EXCELLENT ANSWER

dashboard.py provides the graphical interface, receives user input, coordinates the feature extraction and prediction modules, and displays the final results. File 7 saved_model.pkl Question. What is this file? The trained SVM. Nothing more. Instead of training again, we simply load it. Question. Can we read this file like a text file? No. It stores serialized Python objects. File 8 scaler.pkl Question. Why save the scaler? Because the exact same scaling must be used during prediction. Remember Chapter 16.

PROFESSOR ASKS

Why save the StandardScaler?

EXCELLENT ANSWER

Saving the scaler ensures that future inputs are transformed using exactly the same normalization parameters that were used during model training. File 9 requirements.txt This file contains all Python libraries needed for the project. Example numpy pandas scikit-learn matplotlib tensorflow joblib Question. Why? Because another computer can install everything automatically.

Why This Structure Is Good

Suppose tomorrow you want to replace SVM with Random Forest. Question. Which file changes? Only train_svm.py Everything else stays almost the same. This is called Modular Design.

Data Flow Between Files

Everything works like this. **dataset.csv** — feature_extraction.py. **train_svm.py** — saved_model.pkl. **predict.py** — dashboard.py. Every file has one responsibility. That is good software engineering.

Real Life Analogy

Imagine a restaurant. **Chef** — Cooks food. **Waiter** — Serves food. **Cashier** — Handles payment. **Manager** — Coordinates everyone. Question. Should the cashier cook food? No. Exactly the same principle. Every Python file has its own responsibility.

COMMON MISTAKES

MISTAKE: [X] Putting everything inside one file. Wrong.

MISTAKE: [X] Training inside the dashboard. Wrong.

MISTAKE: [X] Feature extraction inside predict.py. Wrong.

MISTAKE: [X] Saving only the model and forgetting the scaler. Wrong.

REVISION NOTES

Remember the file structure.
dataset.csv — Data.
feature_extraction.py — 17 Features.
train_svm.py — Train SVM.
train_ann.py — Train ANN.
predict.py — Inference.
dashboard.py — GUI.
saved_model.pkl — Trained Model.
scaler.pkl — Normalization.
requirements.txt — Libraries.

MEMORY TRICK

Remember this sentence. "Extract, Train, Save, Predict, Display." Five words. Entire project architecture.

KNOWLEDGE CHECK

Level 1

- What is stored in dataset.csv?
- What is the purpose of feature_extraction.py?
- What does train_svm.py do?
- Why is predict.py separate from training?
- Why is the scaler saved?

Level 2

- Explain the responsibility of each Python file.
- Why is modular design important?
- Why does the dashboard not contain training code?
- Why must both the model and scaler be saved?
- How do all project files interact?

Professor Level

- Explain the complete software architecture of your Python implementation.
- Why is separating training, prediction, and GUI considered good software engineering?
- Describe the data flow through all Python modules.
- If SVM were replaced by another classifier, which files would change?
- How does your project architecture improve maintainability and scalability?
- End of Chapter 23 (Part 1)
- Congratulations.
- You now understand the overall Python project structure.
- This chapter provides the "big picture" of the codebase before diving into individual functions and lines of code.

Author's Note (Important) From this point onward, I cannot accurately invent or explain your exact code line-by-line unless I have the actual source files. Up to now, everything has been based on the project architecture and concepts we developed together. For the remaining code chapters, I need your real implementation so that the explanations exactly match what you wrote. Please upload your project source code (or a ZIP containing it), especially files such as: feature_extraction.py train_svm.py train_ann.py predict.py dashboard.py Any helper modules Once you upload them, I'll continue the Project Bible in the same style and produce:

Chapter 23 Part 2: Every function explained line by line. Chapter 23 Part 3: Every important code block explained. Chapter 24: Dashboard code explained line by line. Chapter 25: Every bug, exception, and troubleshooting case. Chapter 26+: Based on your actual implementation, not a generic template. This way, the remaining chapters will be 100% accurate to your project rather than hypothetical.

PART 2 — day12_build_sweep_dataset.py (Parsing)

Chapter 23 — Complete Python Source Code Explained Part 2 — day12_build_sweep_dataset.py
(Based on your actual source code)

Before Starting

This is the script that started your entire project. Without this file, nothing else exists. No dataset. No features. No SVM. No dashboard. Everything starts here. Goal of this Script Professor asks What is the purpose of day12_build_sweep_dataset.py?

Perfect Answer

This script automatically reads all LTspice parameter sweep files, parses every simulation run, resamples each waveform to a fixed length of 200 points, and combines all circuits and fault conditions into one master dataset for machine learning. Excellent answer.

Complete Pipeline

Read Text Files

Extract Every Sweep

Resample Every Waveform

Convert to 200 Points

Create Dataset

master_dataset_sweep.csv Everything in one script.

```
Import Section
import os
```

Question Why import os? Because the program must find files inside folders. Examples os.path.join() os.path.isfile() os.path.getsize() Without os Python cannot easily navigate directories. Next

```
import numpy as np
```

Why? Because waveforms are numerical arrays. Examples np.linspace() np.array() np.clip() Almost every scientific project uses NumPy. Next

```
import pandas as pd
```

Question Why Pandas? Because the final dataset must become a CSV file. Pandas creates tables very efficiently. Next

```
from scipy.interpolate import interp1d
```

This is very important. Question Why? Because every LTspice waveform contains different numbers of samples.

Machine Learning

cannot train on variable-length inputs. So we convert every waveform to exactly 200 points. That process is called Interpolation. Configuration Section BASE =. Question What is BASE? Simply the folder containing all sweep files. Think of it as the project root. Next T_END Question Why is this dictionary needed? Different circuits were simulated for different durations. For example **RC** — 0.095 seconds. while **Sallen-Key** — 0.020 seconds. The script needs to know the correct time range before resampling. Next N_PTS = 200 Question Why 200? Because every waveform must have exactly 200 voltage values. If one waveform has 1,000 samples and another has 400, the ML model cannot accept them directly. So every waveform becomes 200 samples. This creates uniform input dimensions. Next

CIRCS Stores all circuit types used in the project. Examples RC RLC RC_LADDER Sallen_Key The script loops through each circuit automatically. Next FAULTS Stores all fault types to search. normal drift open short Again, instead of writing the same code four times, a loop handles everything. Good programming practice. Function 1 find_sweep_file() Question Why create this function? Because different LTspice files may have slightly different names. For example rc_open_sweep.txt or RC_open_sweep or Sallen_key_open_sweep Instead of hardcoding every filename, the function tries multiple possibilities. Very smart design. Inside stems = ['.'] This creates possible filenames. Then exts = ['.txt',''] Meaning search both with and without the extension. Then os.path.isfile() Question Why? To check whether the file exists. Then os.path.getsize() Question Why? Sometimes an empty file exists. Checking file size avoids processing invalid files. Professor asks Why check file size?

Perfect Answer

Because an empty or corrupted file may exist. Verifying the file size ensures that only valid sweep files are processed. Function 2 parse_step_file() This is the heart of the script. Question What does it do? It reads one LTspice sweep file and separates every parameter sweep into individual waveforms. Remember how LTspice works. One text file contains many simulations. Example

Step 1

Waveform

Step 2

Waveform

Step 3

Waveform The parser must detect where one run ends and another begins. That's exactly what parse_step_file() does. Inside runs = [] Question Why? This list stores every completed waveform. Then cur_t cur_v These store the current waveform being read. Think of them as temporary storage. Then flush() One of the most important ideas in this script. Question What does flush() do? Answer It saves the completed waveform into the master list before starting the next waveform. Without it, previous simulations would be lost. Professor asks Why was flush() necessary?

Perfect Answer

Because one LTspice file contains multiple simulation runs. The flush function safely stores the completed waveform before beginning the next run. Next The code looks for

Step Information

Question Why? Because LTspice writes this line before every parameter sweep. This tells our program that a new simulation has started. Then cur_p stores the parameter value used for that simulation. Example

```
R = 100 Ω
```

```
R = 120 Ω
```

```
R = 150 Ω
```

Each sweep has its own parameter value. The script records it. Next The parser ignores Time Step Why? These are header lines. Not actual waveform data. Then parts = line.split() Question Why? Each line contains something like 0.0000 0.000 0.0010 0.023 0.0020 0.145 Splitting separates Time and Voltage. Then float(parts[0]) float(parts[1]) Convert text into numbers. Without this conversion, Python treats them as strings.

Machine Learning

cannot work with strings. Next cur_t.append() cur_v.append() These gradually build one waveform. Example **Time** — [0. 0.001 0.002.] **Voltage** — [0. 0.02 0.14.] Why This Script Is Important This file is much more than a "CSV creator." It solves several engineering problems: Finds the correct LTspice

sweep files automatically. Handles different filename formats. Reads multiple parameter sweeps from a single file. Separates each simulation run correctly. Extracts both time and voltage values. Stores parameter values for each run. Prepares the data for uniform resampling. Without this preprocessing pipeline, the downstream machine learning workflow could not operate on consistent inputs.

REVISION NOTES

Remember this sequence:

Locate Files — Open File

Read Line by Line — Detect New Sweep

Store Time & Voltage — Flush Completed Waveform

Repeat

This is the first half of your dataset generation pipeline.

End of Chapter 23 (Part 2) You now understand the first half of `day12_build_sweep_dataset.py`, including the imports, configuration, file discovery, and LTspice sweep parsing.

Author's Note The next continuation will cover the second half of the same script, including: `resample()` explained line by line. Why interpolation is required. Building the final dataset row by row. Creating `master_dataset_sweep.csv`. Common bugs and edge cases. Professor-level cross questions specifically about dataset generation. This will complete the explanation of your first source file.

PART 3 — day12_build_sweep_dataset.py (Resampling)

Chapter 23 — Complete Python Source Code Explained Part 3 — day12_build_sweep_dataset.py (Resampling & Dataset Creation) (Based on your actual source code)

Before Starting

In Part 2,

we learned how your script reads LTspice sweep files. Now, we'll study how those raw waveforms become the final dataset used by machine learning. This is the second half of the script. Where Are We? Our pipeline currently looks like **LTspice Sweep File — Parser**.

Time Array

+

Voltage Array

Now we need to convert every waveform into exactly 200 voltage samples. Function `resample()` Your code defines

```
def resample(t, v, t_end, n=N_PTS):
```

Question. Why create this function? Because every LTspice simulation does not contain the same number of samples. Example

Waveform A

820 points **Waveform B** — 931 points. **Waveform C** — 1045 points. Question. Can SVM train using different-length inputs? No.

Machine Learning

requires fixed-length feature vectors.

Step 1 — Creating a Uniform Time Grid

Your code uses

```
grid = np.linspace(0.0, t_end, n)
```

Question. What is `np.linspace()`? It creates evenly spaced points. Example Suppose `t_end = 10` seconds `n = 6` The grid becomes 0 2 4 6 8 10 In your project, instead of 6 points, you generate 200 points. Exactly the same idea.

PROFESSOR ASKS

Why use `np.linspace()`?

EXCELLENT ANSWER

`np.linspace()` generates a uniform time grid so that every waveform is represented using the same number of equally spaced sampling points before machine learning.

Step 2 — Why Clip the Grid?

Your code then executes

```
grid = np.clip(grid, t.min(), t.max())
```

Question. Why? Suppose one waveform ends slightly early. Example **Expected** — 0.095 s. **Actual** — 0.09498 s. Without clipping, some grid points would lie outside the waveform. Interpolation would then need to guess values. That process is called extrapolation. We don't want that. So `np.clip()` forces

every grid point to remain inside the valid range.

PROFESSOR ASKS

Why is clipping necessary?

EXCELLENT ANSWER

Different simulation runs may have slightly different end times. Clipping ensures that interpolation occurs only within the available data range, avoiding extrapolation errors.

Step 3 — Linear Interpolation

Your code uses `interp1d(.)` with `kind="linear"` Question. What does linear interpolation mean? Suppose we know only two points. Time 0 s — 0 V. Time 1 s

10 V

Question. What is the voltage at 0.5 s? Linear interpolation draws a straight line. Answer

5 V

Simple.

Real Life Analogy

Imagine two cities.

City A

0 km

City B

100 km Question. Where is 50 km? Exactly halfway. Interpolation works the same way.

Step 4 — Why `bounds_error=False`?

Your code contains `bounds_error=False` Question. Why? Normally, if interpolation receives a point outside the valid range, Python raises an error. Your code prevents that. Instead, it continues safely.

Step 5 — `fill_value`

Your code contains `fill_value=(v[0], v[-1])` Question. What does this do? Suppose a point falls before the waveform. Return the first voltage. Suppose it falls after the waveform. Return the last voltage. This makes the interpolation robust.

PROFESSOR ASKS

Why use `fill_value`?

EXCELLENT ANSWER

If interpolation encounters values outside the valid waveform range, the first or last available voltage is used instead of producing an error. Building the Dataset Now the main loop starts. Your code creates `rows = []` `sid = 0` Question. What is `rows`? A list that stores every sample before creating the DataFrame. Question. What is `sid`? Sample ID. Every waveform receives a unique number. Example

Sample 0

Sample 1

Sample 2

.

```
Sample 1018
Loop 1
for circ in CIRCS:
```

Question. Why? To process every circuit. Example **RC** — Done. **RLC** — Done. **SALLEN_KEY** — Done. RC_LADDER Automatic.

Loop 2

Inside

```
for fault in FAULTS:
```

Now every circuit is combined with every fault. Example **RC** — Normal. **Open** — Short. Drift Nested loops eliminate code duplication. Reading the File Your code calls `find_sweep_file(.)` Then `parse_step_file(.)` Question. What is returned? A list of simulation runs. Each run contains

Parameter Value

Time Array

Voltage Array Resampling Now every run passes through `wave = resample(.)` **Output** — Exactly. 200 voltage values. This is the first time our data has uniform dimensions.

Creating One Dataset Row

Your code creates `row = { sample_id, circuit, fault, param_value }` Question. Why store these? Because

Machine Learning

needs labels, and future analysis needs metadata.

Dictionary Comprehension

One of the smartest lines in your code is

```
row.update({f"v{i}": wave[i] for i in range(N_PTS)})
```

Question. What happens? Instead of writing `v0 v1 v2. v199` manually, Python creates all 200 columns automatically. Result looks like `v0 v1 v2. v199` Much cleaner.

PROFESSOR ASKS

Why use a dictionary comprehension?

EXCELLENT ANSWER

Dictionary comprehension automatically creates the 200 waveform columns, reducing repetitive code and making the implementation more scalable and maintainable. Appending the Row Now `rows.append(row)` `sid += 1` Every waveform becomes one dataset row. Creating the DataFrame Finally

```
df = pd.DataFrame(rows)
```

Question. Why? Because Pandas stores tabular data efficiently. The list of dictionaries becomes a table. Saving the Dataset Your code uses `df.to_csv(.)` Question. What is created? `master_dataset_sweep.csv` This file is the foundation of the entire project. Without it, no feature extraction, no SVM, no dashboard. Verification At the end, your script prints the number of samples per circuit-fault pair using `groupby(.).size()` Question. Why? To verify that every class was generated correctly. This is a simple quality check.

Complete Pipeline

Your entire script now looks like

Locate Sweep Files

Read LTspice Data

Separate Every Simulation

Extract Time & Voltage

Resample to 200 Points

Create Metadata

Build Dataset Rows

Create DataFrame — Save master_dataset_sweep.csv.

Verify Sample Counts

This completes your dataset builder.

Why This Script Is So Important

Most students download datasets. Your project builds its own dataset. That is a major strength. You control Circuit topology. Fault type. Sweep parameters. Dataset quality. Class balance. That gives your project far more engineering depth.

COMMON MISTAKES

MISTAKE: [X] Thinking interpolation changes circuit behavior. It doesn't. It only changes how the waveform is sampled.

MISTAKE: [X] Thinking every LTspice waveform has exactly 200 points. They don't. That's why resampling exists.

MISTAKE: [X] Thinking CSV creation is a trivial step. It isn't. This script establishes the standardized input for the entire machine learning pipeline.

End of Chapter 23 (Part 3) Congratulations. You now fully understand day12_build_sweep_dataset.py from start to finish. You can explain: Why interpolation is needed. Why every waveform becomes 200 points. How rows are constructed. How the final dataset is created. Why this script is the foundation of the entire project. Author's Note The next chapter will begin Chapter 24 — day12b_generate_noise_samples.py. We'll explain: Why you created a noise fault instead of simulating it directly. What Gaussian noise is. Why 25 dB SNR was chosen. Every line of the code. The mathematics behind noise injection. Why the random seed is fixed. How the dataset grows from 815/816 samples to 1019 samples. Why this makes your dataset more realistic. This is one of the most interesting engineering additions in your project because it introduces real-world measurement noise into an otherwise ideal simulation dataset.